

Fase 4 - Spring Boot: Principais Anotações

Esse material mostra como o Spring Boot facilita exatamente a separação em camadas que você praticou na Fase 3 (Controller, Service, Repository), usando anotações. O papel de cada camada continua o mesmo — o que muda é que o Spring passa a “enxergar” e gerenciar essas classes pra você.

1. O que o Spring Boot resolve

Na Fase 3, você criou as classes na mão e conectou tudo manualmente no main (Controller recebendo o Service no construtor, Service recebendo o Repository, etc). Isso se chama **injeção de dependência manual**.

O Spring Boot faz isso automaticamente: ele cria os objetos (chamados de **beans**) e conecta uns nos outros sozinho, seguindo as anotações que você coloca nas classes. Ele também cuida de coisas como subir um servidor web, transformar objetos Java em JSON e vice-versa, e rotear requisições HTTP pro método certo.

2. @SpringBootApplication

Toda aplicação Spring Boot tem uma classe principal marcada com essa anotação:

```
@SpringBootApplication
public class MinhaAplicacaoApplication {
    public static void main(String[] args) {
        SpringApplication.run(MinhaAplicacaoApplication.class, args);
    }
}
```

@SpringBootApplication na verdade agrupa três outras anotações em uma só:

- @Configuration — marca a classe como fonte de configuração.
- @EnableAutoConfiguration — deixa o Spring configurar automaticamente coisas como o servidor web, com base nas dependências que estão no pom.xml.
- @ComponentScan — manda o Spring “escanear” o pacote atual (e os pacotes dentro dele) procurando por classes anotadas com @Component, @Service, @Repository, @Controller, etc, pra gerenciar elas automaticamente.

É esse @ComponentScan que explica por que suas classes de Controller/Service/Repository funcionam mesmo sem você registrar elas manualmente em lugar nenhum — o Spring já sai procurando.

3. As anotações de camada: @Component, @Service, @Repository, @Controller

Lembra da Fase 3, onde cada camada tinha uma responsabilidade? No Spring, cada camada ganha uma anotação correspondente:

```
@Repository
public class ProdutoRepository {
    // ...
}

@Service
public class ProdutoService {
    // ...
}

@RestController
public class ProdutoController {
    // ...
}
```

Só de colocar essas anotações, o Spring já cria e gerencia essas classes como **beans** — objetos que ele mesmo instancia e conecta.

Por que existem quatro anotações diferentes se elas fazem a mesma coisa por baixo? @Component é a anotação genérica-mãe. @Service, @Repository e @Controller são especializações dela — funcionam exatamente igual pro Spring, mas comunicam a **intenção** da classe pra quem está lendo o código. @Repository também tem um efeito colateral extra: ela ativa a tradução de erros de banco de dados específicos pra exceções mais genéricas do Spring.

4. @RestController vs @Controller

- @Controller é usado em aplicações que devolvem uma página HTML (uma “view”, no sentido tradicional do MVC).
- @RestController é usado quando você quer devolver dados diretamente (JSON, geralmente) — é o que vocês usam na empresa pra construir APIs REST.

Na prática, @RestController é @Controller + @ResponseBody — essa segunda anotação diz “o retorno dos métodos aqui não é o nome de uma página, é o dado em si, serializa e manda direto na resposta”.

5. Injeção de dependência: @Autowired vs construtor

Na Fase 3, você conectava as camadas manualmente:

```
ProdutoService service = new ProdutoService(repository);
```

Com Spring, existem duas formas de fazer o framework conectar isso pra você:

Injeção por campo (não recomendada):

```
@Service
public class ProdutoService {
    @Autowired
    private ProdutoRepository repository;
}
```

Injeção por construtor (recomendada):

```
@Service
public class ProdutoService {
    private final ProdutoRepository repository;

    public ProdutoService(ProdutoRepository repository) {
        this.repository = repository;
    }
}
```

Por que construtor é melhor: o campo fica **final** (não pode ser trocado depois de criado), fica explícito quais dependências a classe precisa pra existir, e fica muito mais fácil de testar (você pode criar a classe manualmente passando um repository falso no teste, sem precisar do Spring rodando).

Detalhe importante: desde o Spring 4.3, se a classe tem **só um construtor**, o Spring já injeta automaticamente por ele — nem precisa escrever `@Autowired` em cima do construtor. Então o exemplo acima já funciona exatamente como está, sem nenhuma anotação de injeção visível.

6. Mapeamento de rotas: `@RequestMapping` e as variações

Essas anotações ficam nos métodos do Controller e dizem qual URL e qual verbo HTTP aquele método atende:

```
@RestController
@RequestMapping("/produtos")
public class ProdutoController {

    private final ProdutoService service;

    public ProdutoController(ProdutoService service) {
        this.service = service;
    }

    @GetMapping
    public List<Produto> listar() {
        return service.listarTodos();
    }

    @GetMapping("/{id}")
    public Produto buscarPorId(@PathVariable Long id) {
        return service.buscarPorId(id);
    }
}
```

```

    }

    @PostMapping
    public void cadastrar(@RequestBody Produto produto) {
        service.cadastrar(produto);
    }

    @PutMapping("/{id}")
    public void atualizar(@PathVariable Long id, @RequestBody Produto produto) {
        service.atualizar(id, produto);
    }

    @DeleteMapping("/{id}")
    public void remover(@PathVariable Long id) {
        service.remover(id);
    }
}

```

- `@RequestMapping("/produtos")` na classe define um prefixo comum pra todas as rotas dali.
- `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping` são atalhos pra `@RequestMapping` já dizendo qual verbo HTTP (GET busca, POST cria, PUT atualiza, DELETE remove).

7. Pegando dados da requisição: `@RequestBody`, `@RequestParam`, `@PathVariable`

- **`@RequestBody`**: pega o corpo da requisição (geralmente um JSON) e transforma automaticamente num objeto Java. Usado em POST/PUT, quando o cliente está mandando um objeto inteiro.
- **`@PathVariable`**: pega um valor que faz parte da própria URL. Em `/produtos/5`, o 5 é capturado com `@PathVariable Long id`.
- **`@RequestParam`**: pega um valor passado como parâmetro de query string, tipo `/produtos?nome=Mouse`.

```

@GetMapping("/buscar")
public List<Produto> buscarPorNome(@RequestParam String nome) {
    return service.buscarPorNome(nome);
}
// chamada: GET /produtos/buscar?nome=Mouse

```

8. `@Entity`, `@Id` e `@GeneratedValue`

Essas três já são do JPA (a especificação que o Spring usa pra mapear classes Java em tabelas de banco de dados), não do Spring em si — mas aparecem juntas o tempo todo:

```

@Entity
public class Produto {

```

```

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

private String nome;
private double preco;

// getters e setters
}

```

- @Entity diz “essa classe representa uma tabela no banco de dados”.
- @Id marca qual atributo é a chave primária.
- @GeneratedValue diz que o valor do id é gerado automaticamente (geralmente pelo próprio banco, auto-incremento), então você não precisa definir ele na mão ao criar um novo registro.

Exercício 4.1 — Identificar cada anotação Sem consultar o material, escreva do zero (numa folha ou num editor de texto simples) o que cada uma dessas anotações faz, em uma frase: @RestController, @Service, @Repository, @Autowired, @RequestBody, @PathVariable, @Entity. Depois confira com o material e corrija o que errou.

Exercício 4.2 — Construir a API de Produtos com Spring Boot Pegue o exercício da Fase 3 (Cliente com Controller/Service/Repository em Java puro) e recrie ele agora como uma API Spring Boot de verdade: 1. Crie um projeto novo no IntelliJ (ou reaproveite o da Fase 0), adicionando as dependências spring-boot-starter-web e spring-boot-starter-data-jpa no pom.xml (revise a Fase 2 se precisar lembrar como adicionar dependência). 2. Crie a entidade Cliente com @Entity, @Id, @GeneratedValue, e os atributos nome e idade. 3. Crie ClienteRepository como uma interface que estende JpaRepository<Cliente, Long> (isso já dá métodos prontos de salvar/buscar/listar, sem você escrever nada dentro). 4. Crie ClienteService com @Service, injeção por construtor, e a mesma regra de negócio da Fase 3 (não aceitar cliente menor de 18 anos). 5. Crie ClienteController com @RestController, @RequestMapping("/clientes"), e os endpoints de cadastrar (POST) e listar (GET). 6. Rode a aplicação e teste os endpoints (pode usar o Postman, o Insomnia, ou até a aba de HTTP client do próprio IntelliJ).

Exercício 4.3 — Onde entra cada anotação Depois de terminar o exercício 4.2, escreva um resumo curto (5-6 linhas) explicando: qual anotação faz o Spring “achar” sua classe sozinho, qual faz a injeção de dependência funcionar sem new, e qual transforma o JSON que chega na requisição em objeto Java.

O que vem a seguir

Na Fase 5, você vai conhecer o Lombok — que elimina boa parte do código repetitivo (getters, setters, construtores) que você já escreveu à mão em todas as fases anteriores. Faz sentido ver isso por último: agora você já sabe exatamente o que esse boilerplate faz, então vai entender o que o Lombok está economizando pra você, em vez de só decorar as anotações dele.